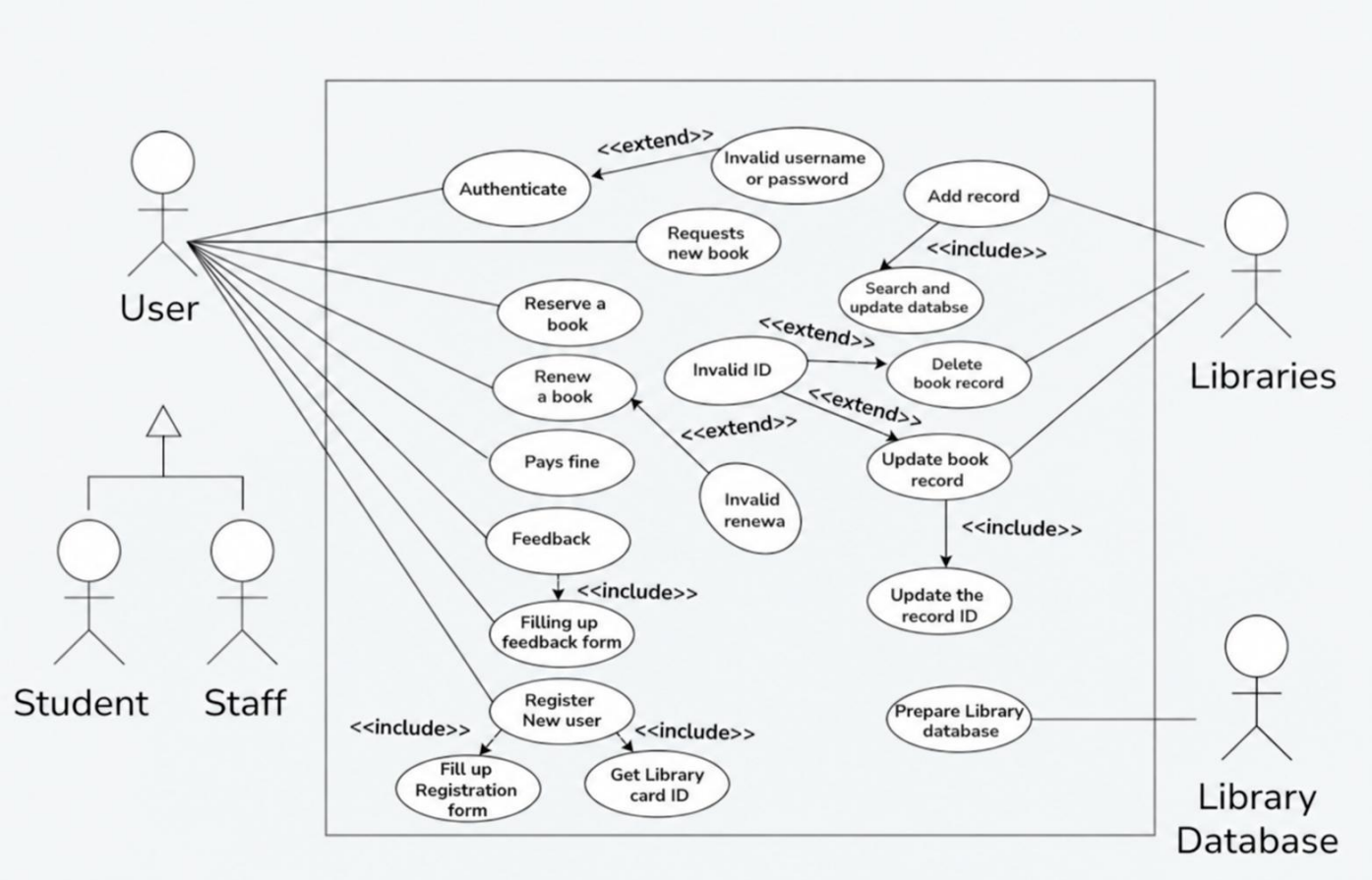
1. **Draw the Use Case Diagram for a Library Management System.**



Library Management System: Use Case Diagram

1. **Actors**

1. User: General actor who performs common library operations.
2. Student: Inherits the use cases of User.
3. Staff: Inherits the use cases of User.
4. Librarian: Manages book records and database operations.
5. Library Database: Supports database operations (if modeled as an external system).

2. **Use Cases**

User-side:

- Authenticate
- Invalid Username or Password (<<extend>>)
- Request New Book
- Reserve a Book
- Renew a Book
- Invalid Renewal (<<extend>>)
- Pay Fine
- Submit Feedback (<<include>> Fill Feedback Form)
- Register New User (<<include>> Fill Registration Form and Generate Library Card ID)

Librarian-side:

- Add Record (<<include>> Search Database)
- Delete Book Record
- Update Book Record (<<include>> Update Record ID)
- Invalid ID (<<extend>> Delete/Update Book Record)

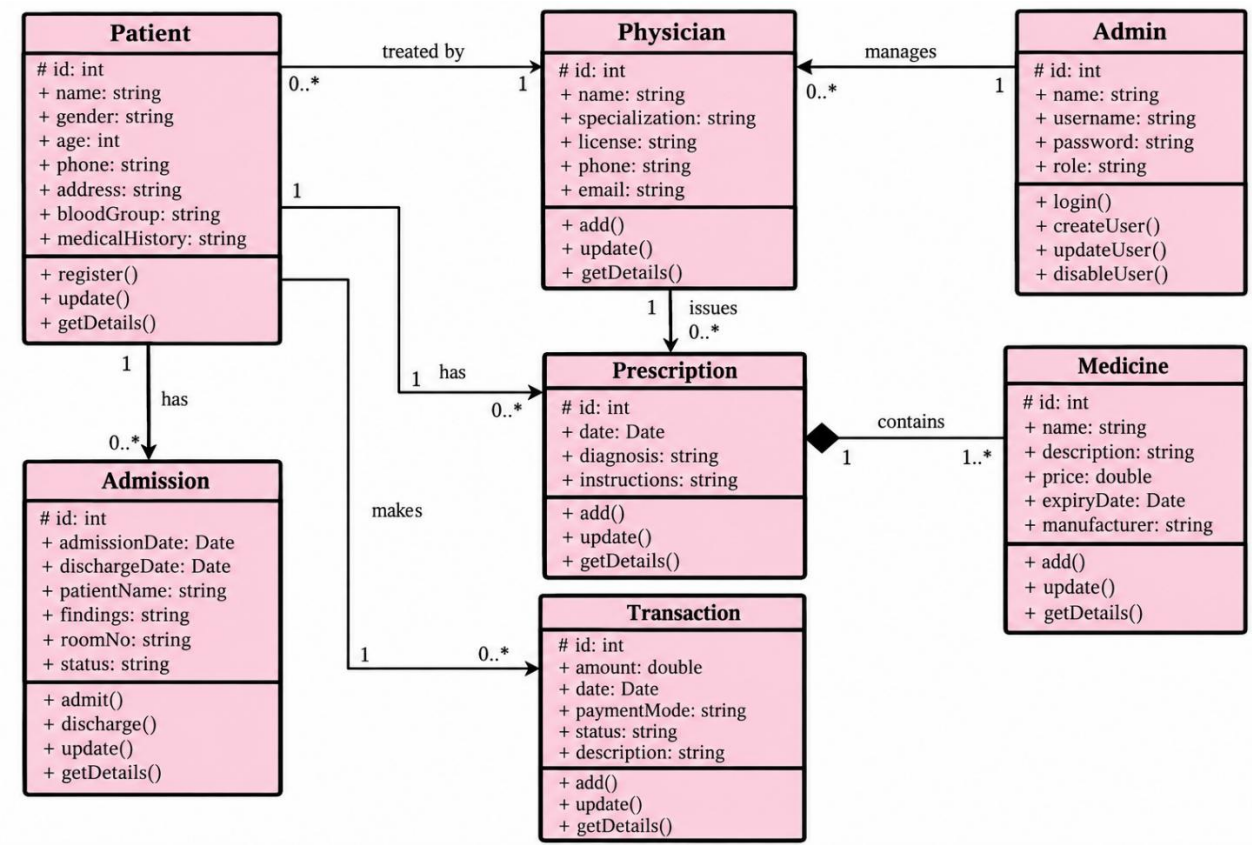
Database-side:

- Prepare Library Database
- Retrieve Records
- Update Records

3. Relationships

- Generalization: Student and Staff inherit from User.
- <<include>>: Mandatory functionality.
- <<extend>>: Conditional or optional functionality.
- Association: Connects actors to their use cases.

2. **Draw the Class Diagram for a Hospital Management System.**



Class Diagram for Hospital Management System

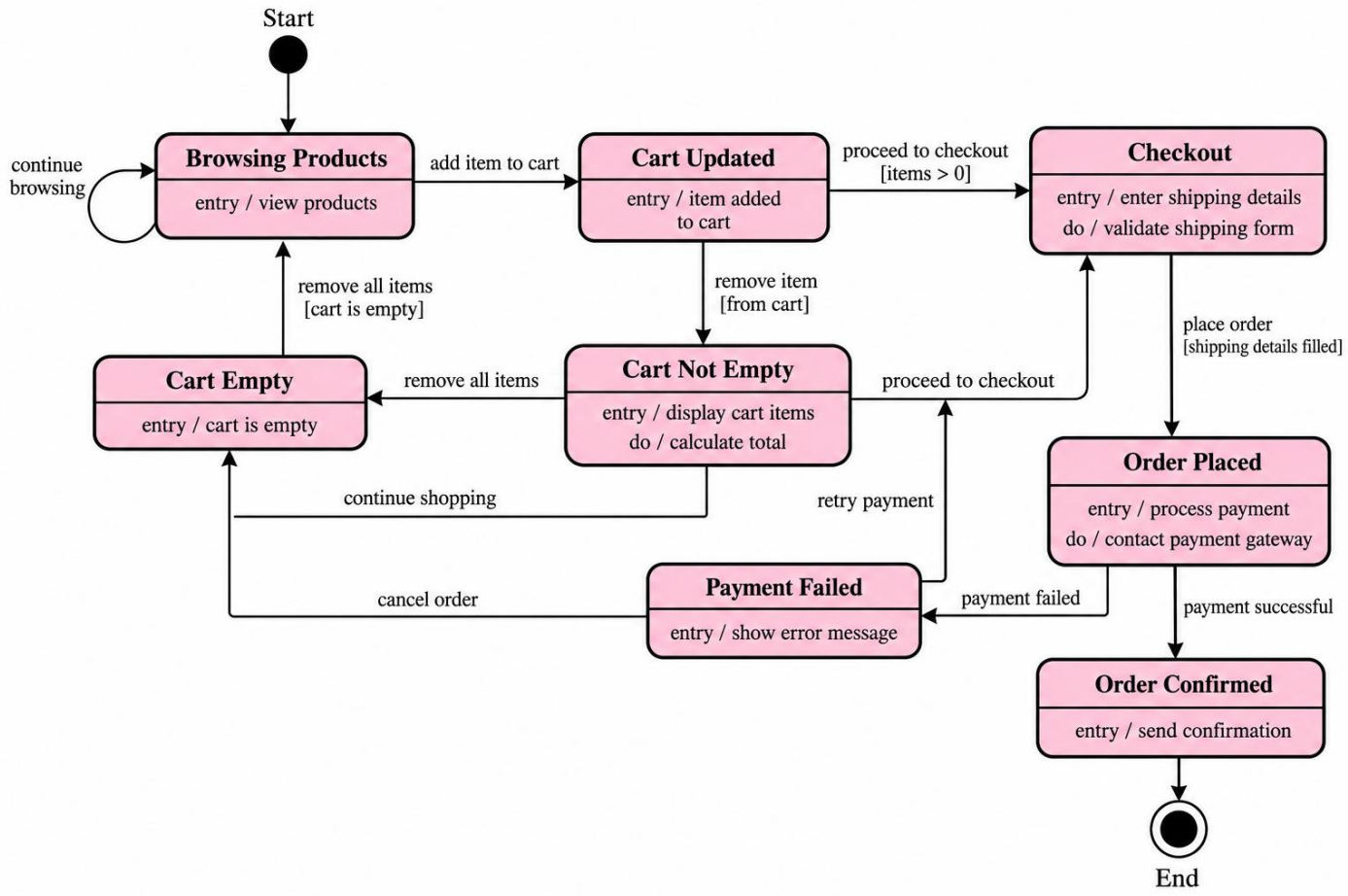
1. **Classes**

1. Patient: Stores patient details such as name, age, gender, contact information, blood group, and medical history. It provides methods to register, update, and retrieve patient details.
2. Physician: Stores doctor details, specialization, license number, phone, and email. It provides methods to add, update, and retrieve physician details.
3. Admin: Manages users and their accounts. It stores administrator details and provides methods to log in, create users, update users, and disable users.
4. Admission: Stores admission date, discharge date, patient name, findings, room number, and admission status. It provides methods to admit, discharge, update, and retrieve admission details.
5. Prescription: Stores prescription ID, date, diagnosis, and instructions. It provides methods to add, update, and retrieve prescription details.
6. Medicine: Stores medicine ID, name, description, price, expiry date, and manufacturer. It provides methods to add, update, and retrieve medicine details.
7. Transaction: Stores transaction ID, amount, date, payment mode, status, and description. It provides methods to add, update, and retrieve transaction details.

2. **Relationships**

- Patient–Physician: Patients are treated by physicians.
- Patient–Admission: A patient can have multiple admissions.
- Patient–Prescription: A patient can have multiple prescriptions.
- Patient–Transaction: A patient can make multiple transactions.
- Physician–Prescription: A physician can issue multiple prescriptions.
- Prescription–Medicine: A prescription contains one or more medicines.
- Admin–Physician: The administrator manages physician accounts.

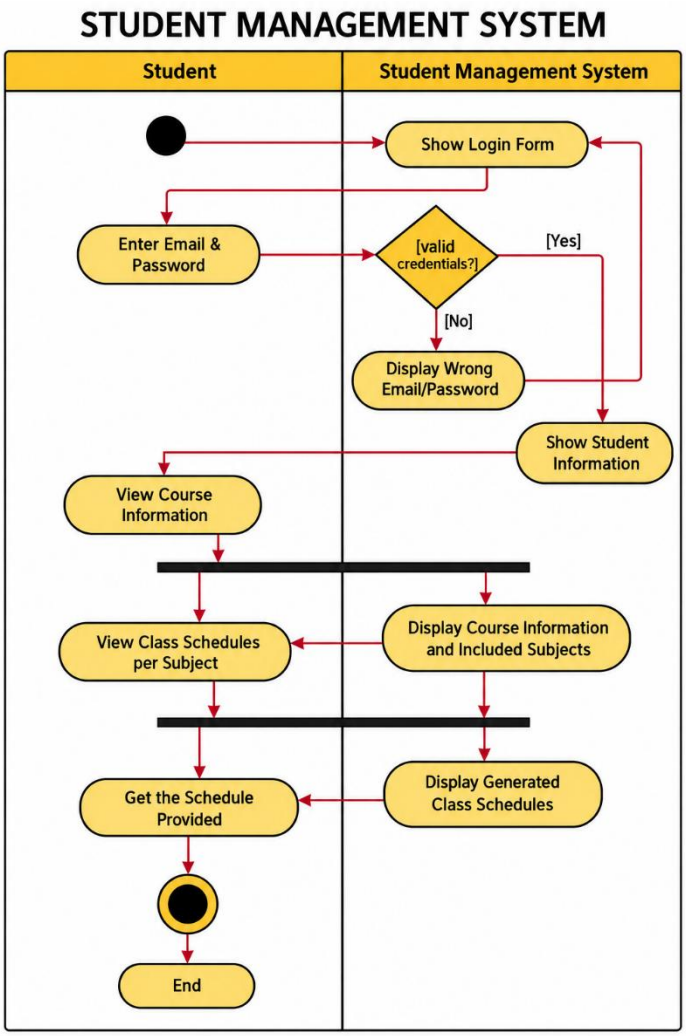
3. **Draw the Statechart Diagram for a Online shopping cart**



This UML Statechart Diagram shows how an online shopping cart works, from selecting products to placing an order.

The customer first browses products and adds items to the cart. They can remove items, proceed to checkout, and enter shipping details. After placing the order, the system processes the payment. If payment succeeds, the order is confirmed; if it fails, the customer can retry payment or cancel the order and return to browsing products.

4. **Draw the Activity Diagram for a student Management System.**



5. **Estimate the development effort and time for a project of size 20 KLOC using the Basic COCOMO model. Assume the project is Organic, with a = 2.4, b = 1.05, c = 2.5, d = 0.38. Calculate the estimated project cost if the development cost is ₹30,000 per person-month.**

Given:

- Project size = 20 KLOC
- Project type = Organic
- $a = 2.4,\ b = 1.05,\ c = 2.5,\ d = 0.38$
- Cost per person-month = ₹30,000

1. Calculation of Development Effort (E)

Formula:

$$E = a \times (KLOC)^b$$

Substituting the values:

$$E = 2.4 \times (20)^{1.05}$$

$$E = 2.4 \times 23.42$$

$$\boxed{E = 56.21 \text{ Person-Months}}$$

2. Calculation of Development Time (D)

Formula:

$$D = c \times (E)^d$$

Substituting the values:

$$D = 2.5 \times (56.21)^{0.38}$$

$$D = 2.5 \times 4.61$$

$$\boxed{D \approx 11.53 \text{ Months}}$$

3. Calculation of Project Cost

Formula:

$$\text{Project Cost} = E \times \text{Cost per Person-Month}$$

Substituting the values:

$$\text{Project Cost} = 56.21 \times 30,000$$

$$\boxed{\text{Project Cost} \approx ₹16,86,300}$$

Final Result

1. **Development Effort:** 56.21 Person-Months
2. **Development Time:** 11.53 Months
3. **Estimated Project Cost:** ₹16,86,300

6. **A college plans to develop a Student Management System to manage student registration, attendance, examination results, and fee details.**

1. **Identify the major project activities and assign suitable durations.**
2. **Define task dependencies and prepare a project schedule using a Gantt chart.**

Answer

1. **Major Project Activities and Durations**

The following table shows the major activities required to develop the Student Management System.

Sr. No.	Project Activity	Duration
1	Requirement Analysis	1 week
2	System Design	2 weeks
3	Database Design	1 week
4	Frontend Development	2 weeks
5	Backend Development	3 weeks
6	Module Integration	1 week
7	Testing and Debugging	2 weeks
8	Deployment	1 week
9	Documentation	1 week

Total project duration: 14 weeks, assuming the activities are completed sequentially.

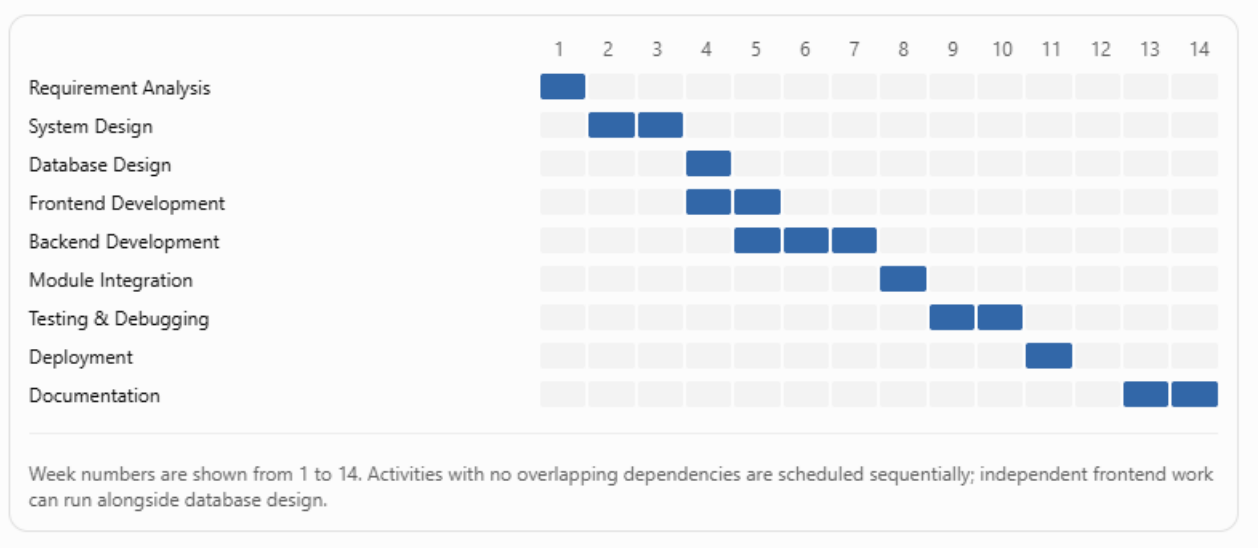
2. **Task Dependencies**

Task dependencies define the order in which project activities must be performed.

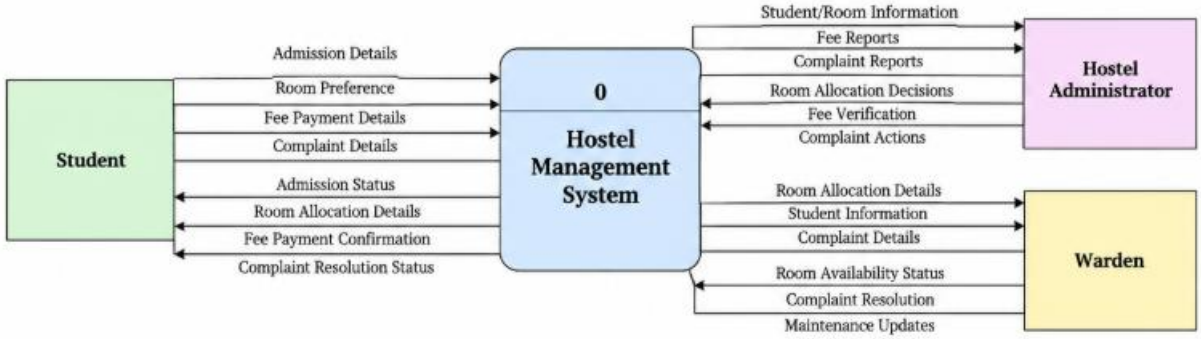
Activity	Depends On
Requirement Analysis	None
System Design	Requirement Analysis
Database Design	System Design
Frontend Development	System Design
Backend Development	Database Design
Module Integration	Frontend and Backend Development
Testing and Debugging	Module Integration
Deployment	Testing and Debugging
Documentation	System Design and development activities

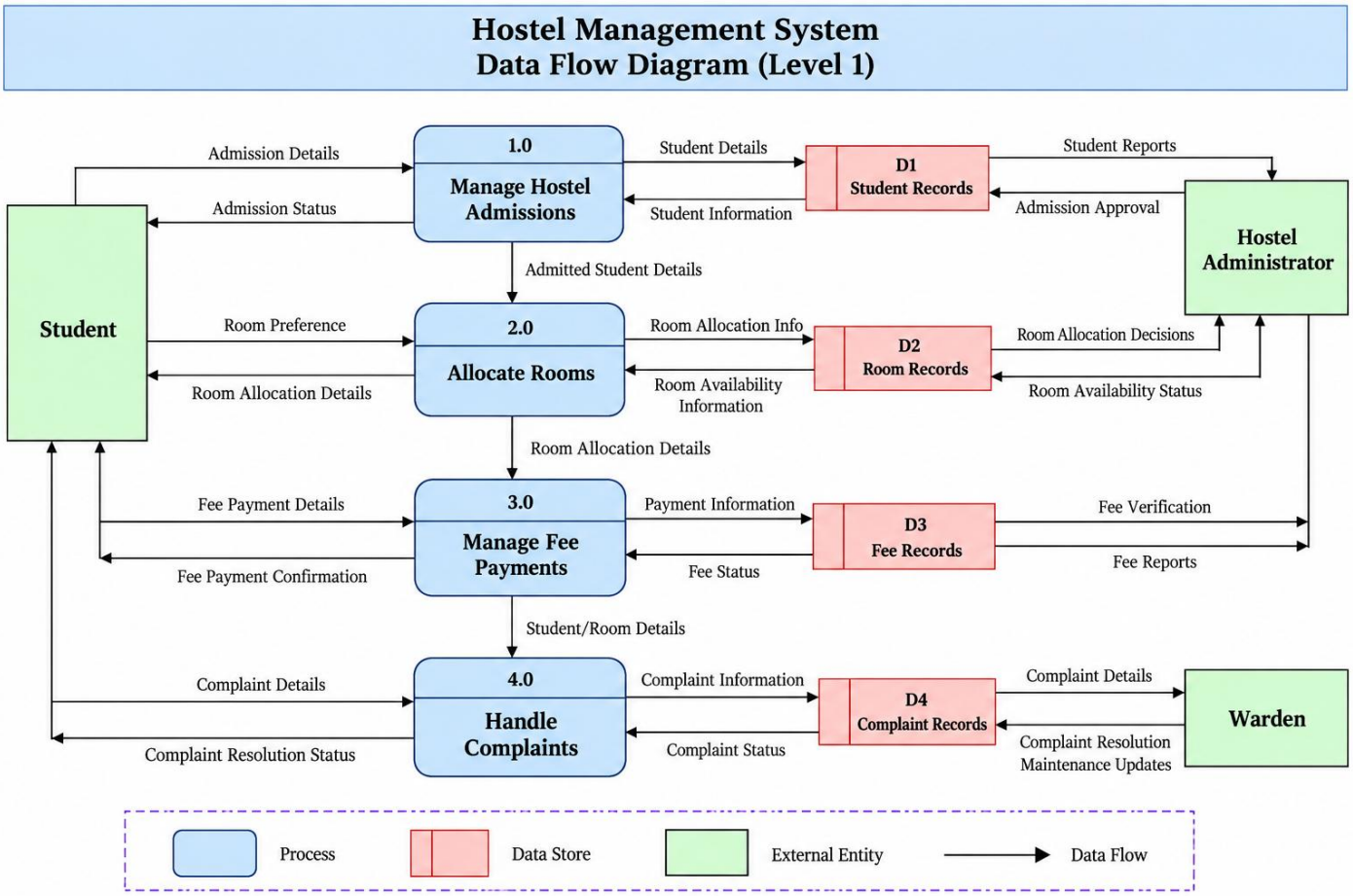
Gantt Chart

The following Gantt chart shows the project schedule. Each block represents one week of work.



7. **Consider a Hostel Management System that manages student hostel admissions, room allocation, fee payments, and complaints. Draw the Context-Level DFD (Level 0) for the given case study. Draw the Level 1 DFD, showing the major processes, data stores, external entities, and data flows. Label all processes, data stores, external entities, and data flows appropriately**

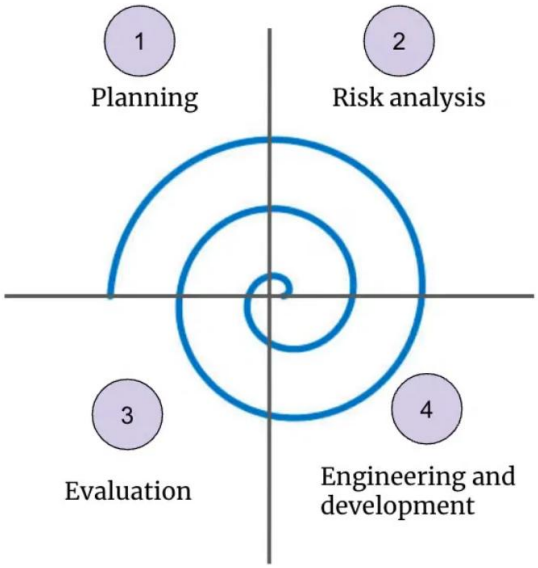




8. **A college plans to develop an Online Examination System to conduct online tests, evaluate answers, generate results, and maintain student records. The requirements may change based on feedback from faculty and students. Study the given case study and identify a suitable software process model.**

Answer: Spiral Model

1. The Spiral Model is an iterative software development model proposed by Barry Boehm in 1986. It combines the features of the Waterfall Model and the Prototyping Model. Software is developed in repeated cycles called spirals, with each spiral consisting of four phases.
2. **Diagram**



4 phases of the spiral model

3. **Four Phases of the Spiral Model**

1. Planning: Define objectives and gather requirements.
2. Risk Analysis: Identify risks and find suitable solutions.
3. Engineering and Development: Design, code, and test the software.
4. Evaluation: Collect feedback from faculty and students.

4. Why the Spiral Model is Suitable

- Changing Requirements: Requirements can be modified based on feedback from faculty and students.
- Risk Management: Risks such as server failure and incorrect evaluation can be identified early.
- Iterative Development: The system can be developed module by module.
- Continuous Feedback: Users can review the system after each cycle.
- Complex System: It supports multiple modules such as login, examinations, evaluation, and result generation.

5. **Application to the Online Examination System**

Spiral	**Modules Developed**
Spiral 1	Student and faculty registration and login
Spiral 2	Question paper display, timer, and answer submission
Spiral 3	Automatic evaluation and result generation
Spiral 4	Faculty dashboard, student records, and final improvements

After each spiral, faculty and students review the system and provide feedback for the next cycle.

6. Why Other Models Are Less Suitable

- Waterfall Model: Difficult to accommodate changing requirements once development progresses.
- V-Model: Changes to requirements can be costly because development and testing phases are closely linked.
- Incremental Model: Works best when major requirements and increments can be planned clearly.
- Agile Model: Can also handle changing requirements, but the Spiral Model is particularly suitable when systematic risk analysis is important.

7. **Advantages of the Spiral Model**

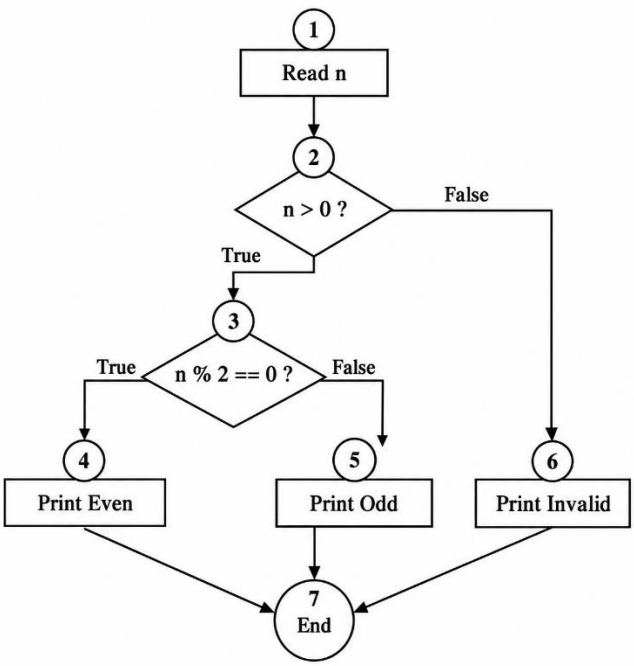
- Handles changing requirements effectively.
- Identifies and manages risks at every cycle.
- Supports iterative development and testing.
- Allows continuous user feedback.
- Suitable for large and complex projects.

9. **Apply Basis Path Testing to the given program and calculate the Cyclomatic Complexity**

```
read n
if (n > 0)
   if (n % 2 == 0)
      print "Even"
   else
      print "Odd"
else
   print "Invalid"
```

Answer

1. **Control Flow Graph (CFG)**



Control Flow Graph (CFG)

Cyclomatic Complexity V(G)

Formula: V(G) = E - N + 2P

Where:

- E = Number of Edges
- N = Number of Nodes
- P = Number of Connected Components (usually 1)

Given Values

- Edges (E) = 8
- Nodes (N) = 7
- P = 1 (single program graph)

Calculation

V(G) = E - N + 2P

V(G) = 8 - 7 + 2(1)

V(G) = 8 - 7 + 2

V(G) = **3**

3. Independent Paths

Since V(G) = 3, there are **3 independent paths.**

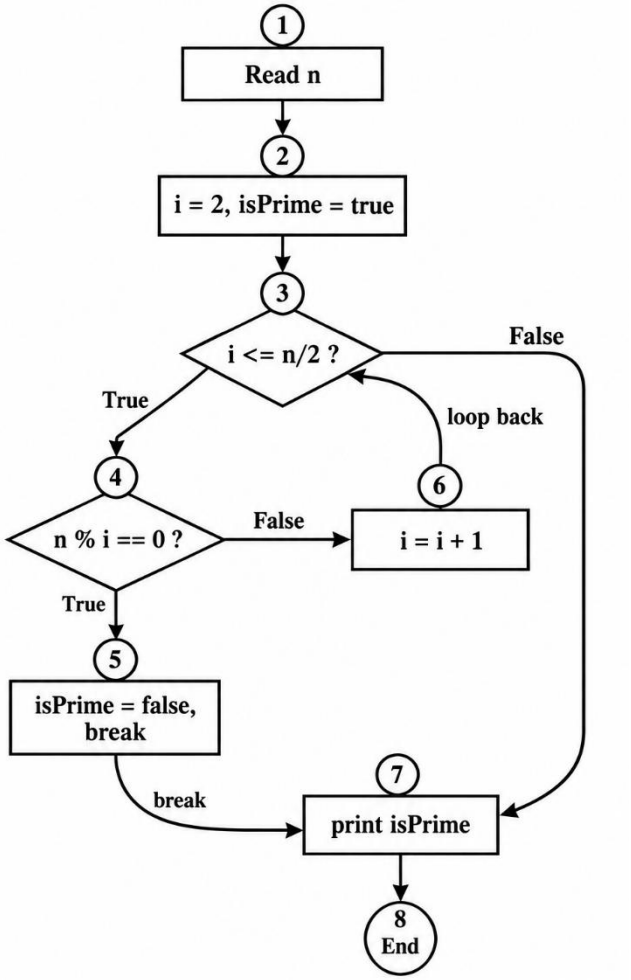
Path	Route	Condition
P1	1 → 2 → 6 → 7	n ⩽ 0
P2	1 → 2 → 3 → 4 → 7	n > 0 and n is even
P3	1 → 2 → 3 → 5 → 7	n > 0 and n is odd

4. Test Cases

Test Case	Input	Path	Expected Output
TC-01	n = -2	P1	Invalid
TC-02	n = 4	P2	Even
TC-03	n = 3	P3	Odd

10. Apply White-box testing and calculate cyclomatic complexity for given program

```
read n
i = 2
isPrime = true
while (i <= n/2)
   if (n % i == 0)
      isPrime = false
      break
   i = i + 1
print isPrime
```



Control Flow Graph (CFG)

E = 9, N = 8, P = 1

Cyclomatic Complexity V(G)

Formula: V(G) = E - N + 2P

Where:

- E = Number of Edges
- N = Number of Nodes
- P = Number of Connected Components (usually 1)

Given Values

- Edges (E) = 9
- Nodes (N) = 8
- P = 1 (single program graph)

Calculation

V(G) = E - N + 2P

V(G) = 9 - 8 + 2(1)

V(G) = 9 - 8 + 2

V(G) = **3**

Step 4: Independent Paths

Since V(G) = 3, there are **3 independent paths.**

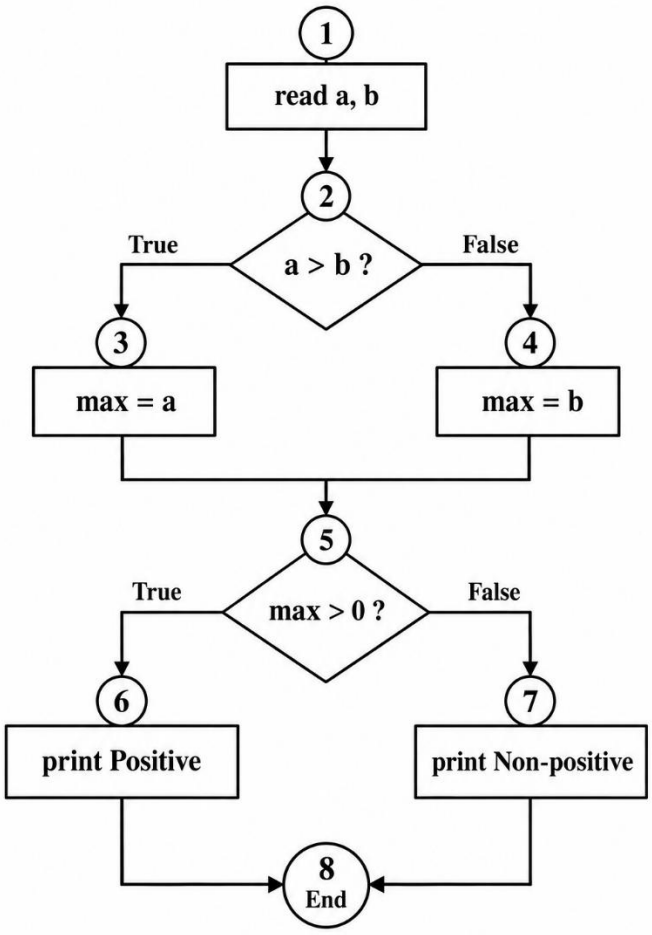
Path	Route	Condition
P1	1→2→3→7→8	while is FALSE from start (n < 2, e.g. n=1)
P2	1→2→3→4→6→3→7→8	while TRUE, if FALSE — loop runs, no factor found (prime)
P3	1→2→3→4→5→7→8	while TRUE, if TRUE — factor found (not prime), break

Step 5: Test Cases

Test Case	Input	Path	Expected Output
TC-01	n = 1	P1	isPrime = **true** (loop skipped)
TC-02	n = 7	P2	isPrime = **true** (prime number)
TC-03	n = 6	P3	isPrime = **false** (not prime)

11. Given the following pseudo-code, apply Basis Path Testing to determine:

```
read a, b
if (a > b)
   max = a
else
   max = b
if (max > 0)
   print "Positive"
else
   print "Non-positive"
```



Control Flow Graph (CFG)

Total Edges (E) = 9, Total Nodes (N) = 8, P = 1

Cyclomatic Complexity V(G)

Formula: V(G) = E - N + 2P

Where:

- E = Number of Edges
- N = Number of Nodes
- P = Number of Connected Components (usually 1)

Given Values

- Edges (E) = 9
- Nodes (N) = 8
- P = 1 (single program graph)

Calculation

V(G) = E - N + 2P

V(G) = 9 - 8 + 2(1)

V(G) = 9 - 8 + 2

V(G) = **3**

3. Independent Paths

Since V(G) = 3, there are **3 independent paths.**

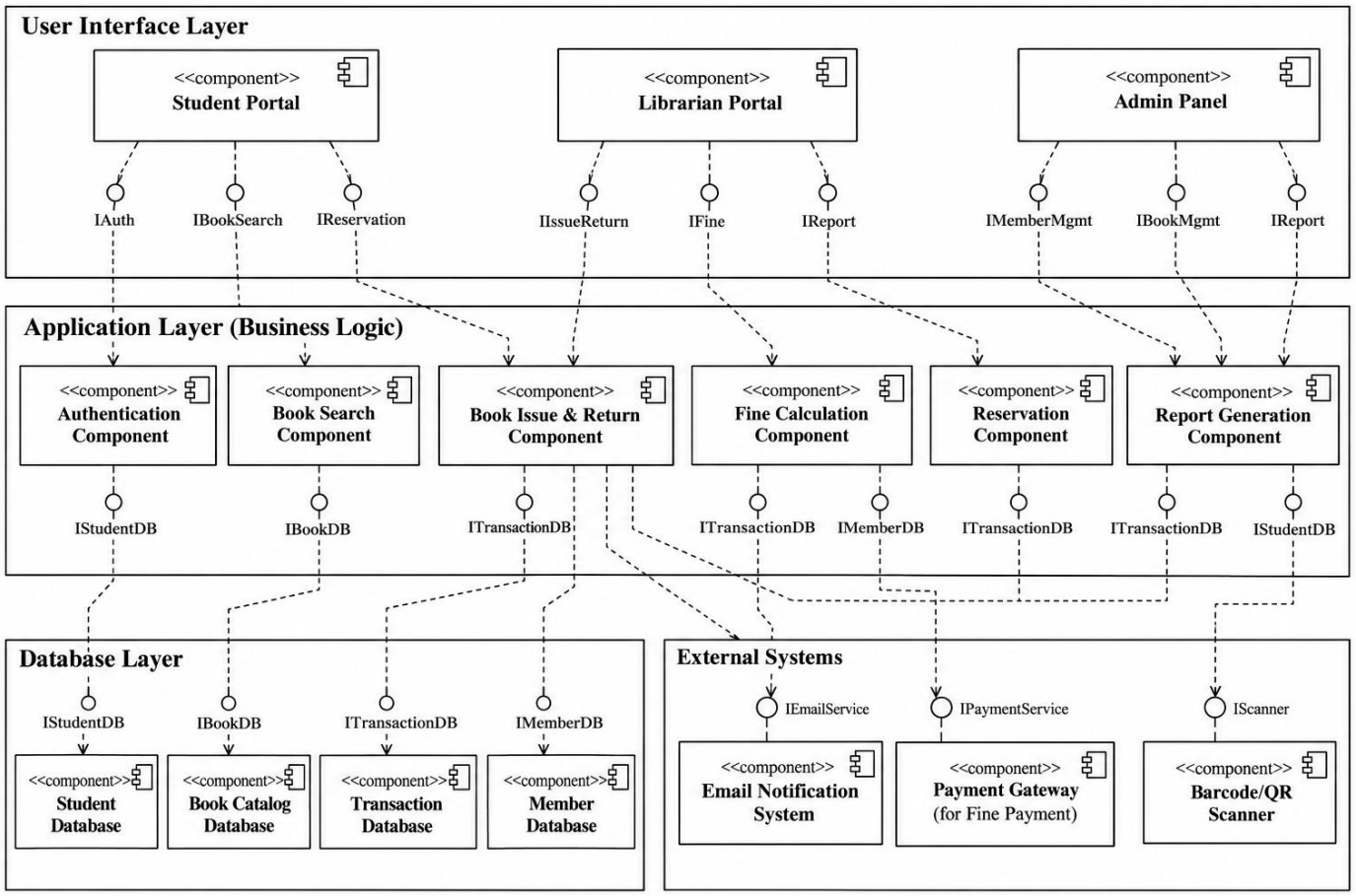
Path	Route	Condition
P1	1→2→3→5→6→8	a > b TRUE, max > 0 TRUE
P2	1→2→3→5→7→8	a > b TRUE, max > 0 FALSE
P3	1→2→4→5→6→8	a > b FALSE, max > 0 TRUE

> **Note:** Path where a > b FALSE and max > 0 FALSE is not a new independent path since V(G) = 3. It is covered by the combination of P2 and P3.

4. Test Cases

Test Case	Input	Path	Expected Output
TC-01	a=5, b=3	P1	max=5, **Positive**
TC-02	a=-2, b=-5	P2	max=-2, **Non-positive**
TC-03	a=3, b=7	P3	max=7, **Positive**

12. Draw the Component Diagram for a Library Management System



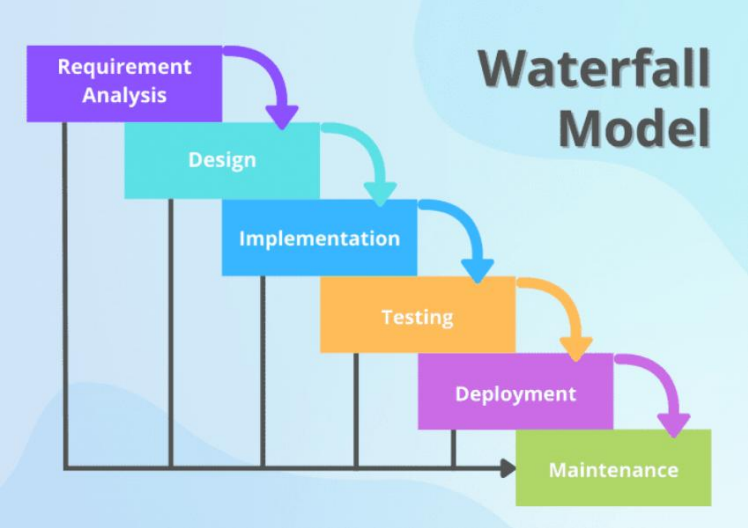
13. **A hospital plans to develop a Hospital Management System to manage patient registration, appointments, billing, laboratory reports, pharmacy, and electronic health records. The hospital requires a secure and reliable system with clearly defined requirements. Major requirement changes are not expected during development. Study the given case study and identify a suitable software process model. Draw the diagram of the selected process model**

Answer: Waterfall Model

1. **Definition**

The Waterfall Model is a linear and sequential software development model. In this model, development proceeds through a series of phases, and each phase is completed before the next phase begins.

2. **Diagram**



3. **Phases of the Waterfall Model**

1. Requirement Analysis: Gather requirements for patient registration, appointments, billing, laboratory reports, pharmacy, and electronic health records.
2. Design: Design the system architecture, database, and user interfaces.
3. Implementation: Develop the different hospital management modules.
4. Testing: Perform unit, integration, and security testing.
5. Deployment: Install and launch the system in the hospital.
6. Maintenance: Fix errors and provide necessary updates after deployment.

4. Why the Waterfall Model is Suitable

- Clearly Defined Requirements: All major modules and functions are identified in advance.
- Stable Requirements: Major changes are not expected during development.
- Security and Reliability: Each phase follows a structured development process.
- Proper Documentation: Every phase produces documentation useful for hospital operations and compliance.
- Easy Management: Progress can be monitored through clearly defined phases and milestones.

5. Why Other Models Are Less Suitable

- Spiral Model: More appropriate when extensive risk analysis and repeated development cycles are required.
- Agile Model: Best suited to projects requiring frequent changes and continuous releases.
- Prototype Model: Useful when requirements are unclear or need clarification.
- Incremental Model: Suitable when the system needs to be delivered in separate functional increments.
- V-Model: Emphasizes corresponding testing activities at each development phase; Waterfall may be simpler when requirements are stable and the project follows a sequential process.

6. Advantages of the Waterfall Model

1. Simple to understand and manage.
2. Each phase has clearly defined deliverables.
3. Provides proper documentation.
4. Makes progress tracking easier.
5. Works well with stable and clearly defined requirements.

7. Disadvantages of the Waterfall Model

1. Difficult to accommodate changes once a phase is completed.
2. Most system testing occurs after implementation.
3. Not suitable for frequently changing requirements.
4. The complete product is generally available late in the development process.

14. A bank plans to develop a Mobile Banking Application with fund transfer, balance enquiry, bill payment, investment services, and biometric authentication. Since financial transactions are involved, the system requires high security, extensive testing, and risk management. Study the given case study and identify a suitable software process model. Draw the diagram of the selected process model

Mobile Banking Application

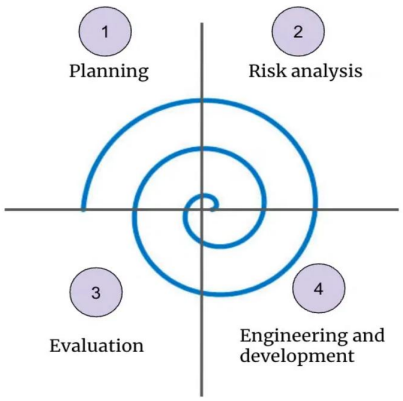
Q. Identify a suitable software process model for the given case study. Justify your answer. Also draw the diagram of the selected model.

Answer: Spiral Model

1. Definition

The Spiral Model is an iterative and risk-driven software development model proposed by Barry Boehm in 1986. It combines the features of the Waterfall Model and the Prototyping Model. Software is developed in repeated cycles called spirals, with each spiral consisting of four phases.

2. Diagram



4 phases of the spiral model

Application of Spirals:

- Spiral 1: Basic application structure and login.
- Spiral 2: Balance enquiry and fund transfer.
- Spiral 3: Bill payment and investment services.
- Spiral 4: Biometric authentication, security testing, and deployment.

3. Four Phases of the Spiral Model

1. Planning: Define objectives, requirements, and constraints.
2. Risk Analysis: Identify security, authentication, and financial risks and find suitable solutions.
3. Engineering and Development: Design, code, and test the current module.
4. Evaluation: Collect feedback from stakeholders and plan the next spiral.

4. **Why the Spiral Model is Suitable**

- High Security: Security risks are identified and addressed in every spiral.
- Risk Management: Financial and technical risks are analyzed before proceeding.
- Extensive Testing: Each developed module is tested before further development.
- Financial Transactions: Fund transfers and bill payments require careful risk evaluation.
- Biometric Authentication: Complex security features can be prototyped and tested.
- Iterative Development: Modules are developed and improved in successive cycles.
- Stakeholder Feedback: Each version can be reviewed before the next cycle begins.

5. **Application to Mobile Banking Application**

Spiral	**Module Developed**	**Risk Handled**
Spiral 1	Application structure and login	Authentication risks
Spiral 2	Balance enquiry and fund transfer	Transaction failure and data security
Spiral 3	Bill payment and investment services	Payment and data integrity risks
Spiral 4	Biometric authentication and final deployment	Authentication failures and security vulnerabilities

6. **Why Other Models Are Less Suitable**

- Waterfall Model: Difficult to accommodate repeated changes and risk-driven iterations.
- Agile Model: Can support frequent changes, but may require additional risk-analysis and documentation practices.
- Incremental Model: Does not inherently provide the same explicit risk-analysis cycle.
- Prototype Model: Focuses on clarifying requirements rather than comprehensive risk management throughout development.
- V-Model: Emphasizes verification and validation but does not inherently include iterative risk analysis.

7. **Advantages of the Spiral Model**

- Identifies and manages risks at every cycle.
- Improves security through repeated testing.
- Supports iterative development and feedback.
- Allows early identification of technical problems.
- Suitable for large, complex, and high-risk applications.
- Supports prototyping of biometric and encryption features.

8. **Disadvantages of the Spiral Model**

- Expensive due to repeated risk analysis.
- Complex to manage.
- Requires experienced risk analysts.
- Time-consuming when many spirals are required.
- Not suitable for small or simple projects.

15. **AI-Based Smart Traffic Management System A city corporation plans to develop an AI-based Smart Traffic Management System to monitor traffic, optimize signal timings, detect accidents, and provide real-time route suggestions. The project involves AI integration, IoT devices, and evolving requirements. Study the given case study and identify a suitable software process model. Draw the diagram of the selected process model**

Answer: Agile Model

1. Definition

The Agile Model is an iterative and incremental software development model based on the Agile Manifesto (2001). Software is developed in small cycles called sprints. It focuses on continuous feedback, collaboration, and adapting to changing requirements.

2. Diagram



3. Phases of the Agile Model

- Requirements: Gather requirements for traffic monitoring, signal optimization, and accident detection.
- Design: Design the system architecture, AI algorithms, and IoT integration.
- Development: Develop the required modules.
- Testing: Test the modules and their integration with sensors and devices.
- Deployment: Deploy the working modules.
- Review: Collect feedback and update requirements for the next sprint.

4. Why the Agile Model is Suitable

- Evolving Requirements: Adapts to changing traffic conditions and user needs.
- AI Integration: Supports continuous improvement of AI models.
- IoT Integration: Allows frequent testing and updates of sensors and signal controllers.
- Fast Delivery: Working modules are delivered in short development cycles.
- Continuous Feedback: The city corporation can review each module and suggest improvements.
- Real-Time Route Suggestions: Supports repeated testing and refinement of route recommendations.
- Team Collaboration: Developers, AI engineers, IoT specialists, and city planners can work together.

5. **Application to the Smart Traffic Management System**

Sprint	Module Developed
Sprint 1	Traffic monitoring using cameras and sensors
Sprint 2	AI-based traffic signal optimization
Sprint 3	Accident detection and alert generation
Sprint 4	Real-time route suggestions using GPS and AI
Sprint 5	AI and IoT integration, performance testing, and deployment

After each sprint, the city corporation reviews the working module and provides feedback for the next sprint.

6. **Why Other Models Are Less Suitable**

- Waterfall Model: Difficult to accommodate frequently changing requirements.
- Spiral Model: Focuses strongly on risk analysis and may involve greater management overhead.
- Incremental Model: Can deliver modules incrementally, but Agile provides a more explicit framework for regular feedback and adaptation.
- Prototype Model: Focuses mainly on clarifying requirements through prototypes.
- V-Model: Changes can be difficult to accommodate because development and testing activities are closely linked.

7. **Advantages of the Agile Model**

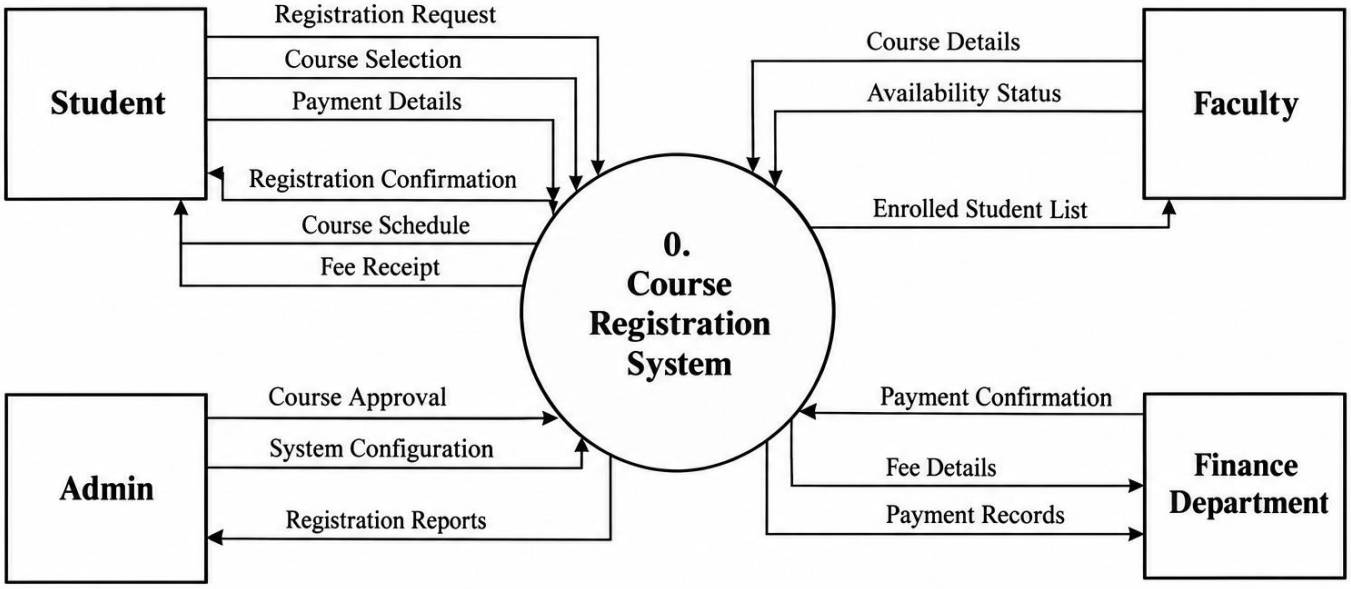
- Handles changing requirements effectively.
- Supports continuous improvement of AI models.
- Enables incremental integration of IoT devices.
- Delivers working software in short cycles.
- Incorporates continuous user feedback.
- Improves collaboration among team members.
- Allows early identification of defects.

8. **Disadvantages of the Agile Model**

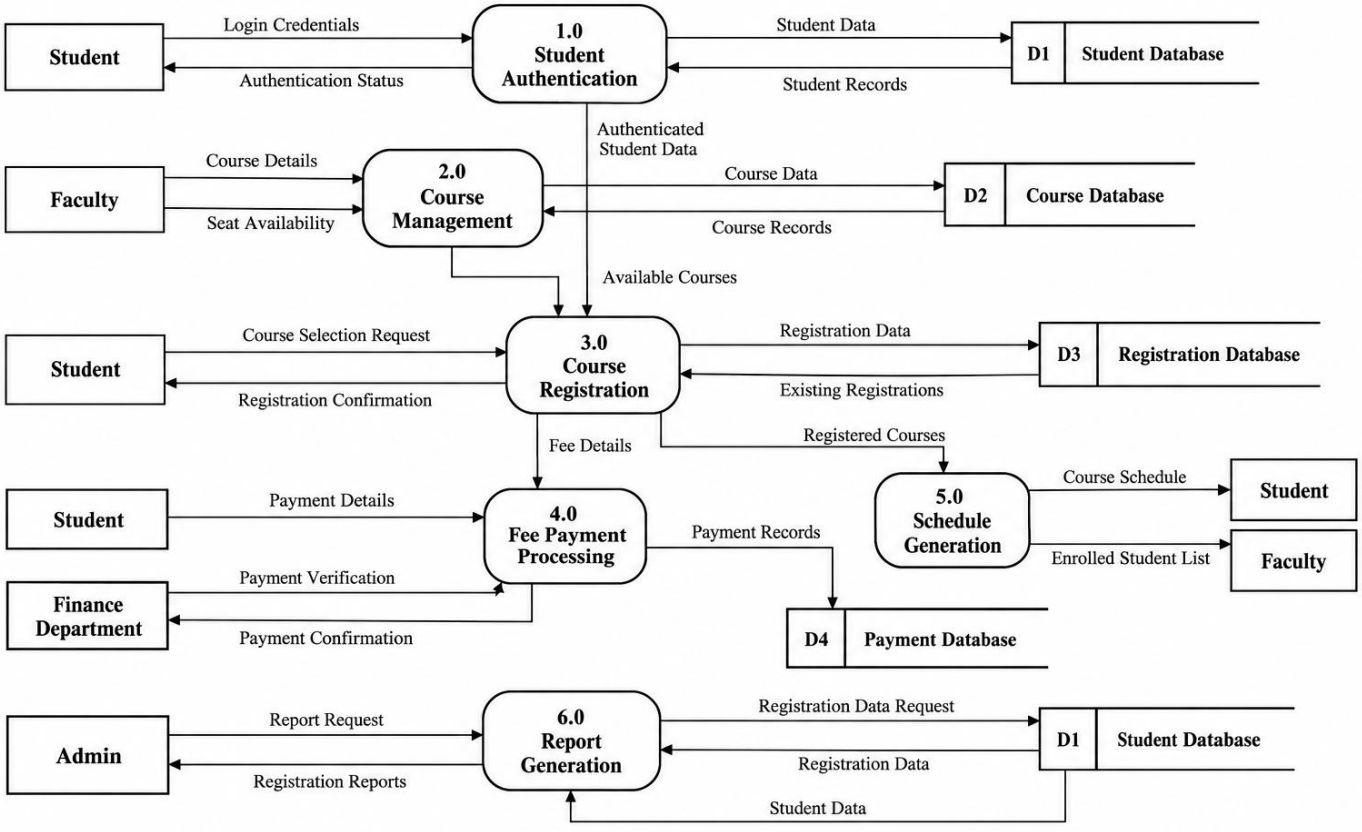
- Total cost and completion time can be difficult to predict.
- Requires skilled and collaborative team members.
- Frequent changes may cause scope creep.
- Requires continuous customer involvement.
- Inadequate documentation can create maintenance difficulties.

16. Consider course registration system Draw the Context-Level DFD (Level 0) for the given case study. Draw the Level 1 DFD, showing the major processes, data stores, external entities, and data flows. Label all processes, data stores, external entities, and data flows appropriately

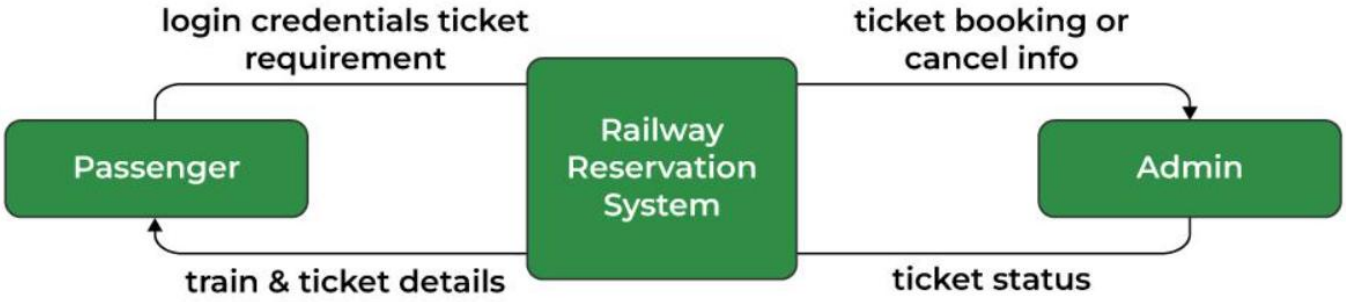
Level 0 DFD - Course Registration System

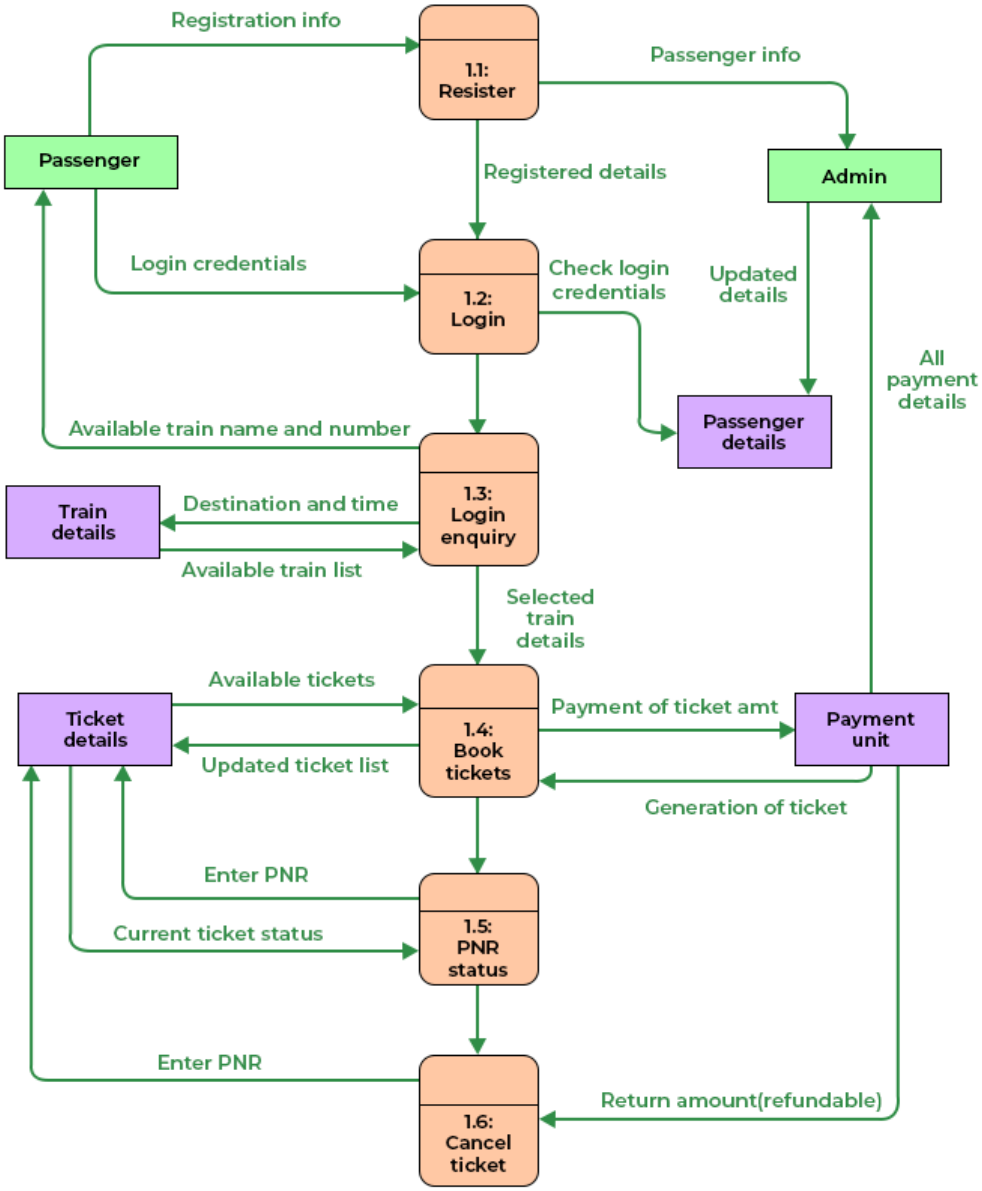


Level 1 DFD - Course Registration System



17. Consider railway reservation system Subject: Software Engineering (SE) Practical / Oral Examination Draw the Context-Level DFD (Level 0) for the given case study. Draw the Level 1 DFD, showing the major processes, data stores, external entities, and data flows. Label all processes, data stores, external entities, and data flows appropriately





18. Online Pharmacy Management System

A healthcare startup plans to develop an Online Pharmacy Management System to enable customers to search medicines, upload prescriptions, place orders, make online payments, and track deliveries. The system will also allow pharmacists to verify prescriptions, manage inventory, and generate sales reports.

The estimated software size is 32 KLOC. Based on previous projects, the organization classifies it as a Semi-Detached project.

The function counts identified are:

- **External Inputs (EI): 18**
- **External Outputs (EO): 14**
- **External Inquiries (EQ): 10**
- **Internal Logical Files (ILF): 8**
- **External Interface Files (EIF): 5**

Assume all functions are of average complexity and the Value Adjustment Factor (VAF) = 1.05.

Questions

- Calculate the Unadjusted Function Points (UFP).
- Calculate the Adjusted Function Points (AFP).
- Calculate the project effort using the Basic COCOMO model. Given: (a = 2.4,\ b = 1.05,\ c = 2.5,\ d = 0.38).
- Find the estimated effort and discuss whether the project is suitable for the selected COCOMO category.

Given Data

- Project: Online Pharmacy Management System
- Project category: Semi-Detached
- Software size: 32 KLOC
- Value Adjustment Factor (VAF): 1.05
- Complexity of all functions: Average

Q1. Calculate the Unadjusted Function Points (UFP)

For average complexity, the standard function point weights are:

Function type	Count	Average weight	Total
External Inputs (EI)	18	4	72
External Outputs (EO)	14	5	70
External Inquiries (EQ)	10	4	40
Internal Logical Files (ILF)	8	10	80
External Interface Files (EIF)	5	7	35
Total UFP			297

Formula:

$$UFP = \sum(\text{Count} \times \text{Weight})$$

$$UFP = 72 + 70 + 40 + 80 + 35$$

$$\boxed{UFP = 297}$$

Q2. Calculate the Adjusted Function Points (AFP)

Formula:

$$AFP = UFP \times VAF$$

Substituting the values:

$$AFP = 297 \times 1.05$$

$$\boxed{AFP = 311.85 \approx 312\ FP}$$

Answer: The Adjusted Function Points are **311.85 FP** (approximately 312 FP).

Q2. Calculate the Adjusted Function Points (AFP)

Formula:

$$AFP = UFP \times VAF$$

Substituting the values:

$$AFP = 297 \times 1.05$$

$$\boxed{AFP = 311.85 \approx 312\ FP}$$

Answer: The Adjusted Function Points are **311.85 FP** (approximately 312 FP).

Q3. Calculate the project effort using the Basic COCOMO model

Given:

- $KLOC = 32$
- $a = 2.4$
- $b = 1.05$
- $c = 2.5$
- $d = 0.38$

Step 1: Calculate effort

The Basic COCOMO effort formula is:

$$E = a \times (KLOC)^b$$

Substituting the values:

$$E = 2.4 \times (32)^{1.05}$$

$$E = 2.4 \times 38.0546$$

$$\boxed{E = 91.33 \text{ Person-Months}}$$

Step 2: Calculate development time

The development time formula is:

$$T = c \times (E)^d$$

$$T = 2.5 \times (91.33)^{0.38}$$

$$\boxed{T = 13.90 \text{ Months}}$$

Step 3: Calculate average team size

$$\text{Team Size} = \frac{E}{T}$$

$$= \frac{91.33}{13.90}$$

$$\boxed{\text{Team Size} \approx 7 \text{ Persons}}$$

Q4. Find the estimated effort and discuss whether the project is suitable for the selected COCOMO category

The estimated effort is **91.33 Person-Months**, with a development time of approximately **13.90 months**.

The **Semi-Detached** category is generally suitable for projects of moderate complexity that involve a mix of experienced and less-experienced developers. An online pharmacy system has several interacting modules, including prescription verification, inventory management, online payments, order tracking, and reporting.

Therefore, the Semi-Detached category is a reasonable choice based on the stated classification. However, the final suitability depends on the team's experience, technical complexity, and project constraints.

Important note: The question provides $a = 2.4$, $b = 1.05$, $c = 2.5$, and $d = 0.38$, and the calculations above use those exact values. These coefficients do not match the standard Basic COCOMO Semi-Detached coefficients, which are $a = 3.0$, $b = 1.12$, $c = 2.5$, and $d = 0.35$. If your examiner expects the standard Semi-Detached coefficients, use those instead.

Final Answers

Question	Answer
1. UFP	297
2. AFP	311.85 FP ≈ 312 FP
3. Estimated effort	91.33 Person-Months
3. Development time	13.90 Months
3. Average team size	Approximately 7 persons
4. Suitability	Semi-Detached is reasonable for the stated project classification.

19. Smart Parking Management System

A municipal corporation plans to develop a Smart Parking Management System that allows users to locate available parking spaces, reserve slots, make online payments, and receive real-time parking updates. Parking administrators can monitor occupancy and generate daily reports.

The estimated project size is **28 KLOC**, and the project is categorized as **Organic**.

Given: (a = 2.4, b = 1.05, c = 2.5, d = 0.38).

Questions

1. Calculate the project effort and development time using the Basic COCOMO model.
2. If the organization's productivity is 600 LOC per person-month, calculate the effort using the LOC method.

Q1. Calculate the project effort and development time using the Basic COCOMO model.

Given:

- Project size = 28 KLOC
- Project type = Organic
- $a = 2.4,\ b = 1.05$
- $c = 2.5,\ d = 0.38$

Step 1: Calculate project effort

Formula:

$$E = a \times (KLOC)^b$$

Substituting the values:

$$E = 2.4 \times (28)^{1.05}$$

$$E = 2.4 \times 32.676$$

$$\boxed{E = 78.42 \text{ Person-Months}}$$

Step 2: Calculate development time

Formula:

$$T = c \times (E)^d$$

Substituting the values:

$$T = 2.5 \times (78.42)^{0.38}$$

$$T = 2.5 \times 5.145$$

$$\boxed{T = 12.86 \text{ Months}}$$

Answer:

- Estimated effort = **78.42 Person-Months**
- Development time = **12.86 Months**

Q2. Calculate the effort using the LOC method.

Given:

- Project size = 28 KLOC
- Productivity = 600 LOC per person-month

Step 1: Convert KLOC into LOC

$$LOC = 28 \times 1000$$

$$LOC = 28000$$

Step 2: Calculate effort

Formula:

$$\text{Effort} = \frac{\text{Total LOC}}{\text{Productivity}}$$

Substituting the values:

$$E = \frac{28000}{600}$$

$$\boxed{E = 46.67 \text{ Person-Months}}$$

Answer: The estimated effort using the LOC method is **46.67 Person-Months**.

Final Answers

Question	Result
Q1: COCOMO effort	78.42 Person-Months
Q1: Development time	12.86 Months
Q2: LOC method effort	46.67 Person-Months

20. Draw Class diagram for Travel Agency system

